

Voxelcraft codebase & documentation audit

Scope: the full tree extracted from `uploads/codebase_dump.txt` (169 files: ~36k lines Rust, ~7.6k lines WGSL in 9 shaders, ~15k lines of docs). **Method:** full `cargo check --all-targets`; every test target executed (debug, and release where timing mattered); all WGSL validated through naga 30 (`wgsl -in`, `Capabilities::all()`); hand cross-checks of Rust↔WGSL layouts, flag values, CLI flags, bind groups, dispatch sizes; every claim in `HANDOFF.md` / `BATCH-LOG.md` that is checkable without a GPU verified against source; all markdown links resolved; doc-vs-code consistency sweeps (probe constants, shadow architecture, menu, CLI help).

Environment caveat: 2 CPU cores, ~2 GB RAM, no GPU. This matters for exactly one finding (§4.1), which is proven environmental rather than a code defect.

1. Real code defects

1.1 `march_chunk` breaks on axis-aligned rays — the root cause of the recorded glass-transport failure

`validation/glass-transport-results.txt` records the known failure:

```
AssertionError: (3.5, 0.0, 512.0, 0.0)
```

The fixture (`validation/check_glass_transport.py`) fires an **exactly axis-aligned ray** `rd = (1, 0, 0)` and expects the transmission leg to reach the target at $t=11.5$. It gets $t=512$ (max distance) and material 0 — a miss. The GLO1 handoff says the glass hypothesis "made no traversal change" and points at "existing DDA reciprocal/step behavior". That pointer is correct, and the defect is still live:

- Batch CLO2 fixed `trace_world` (`src/render/shaders/common.wgsl:2307`): it epsilon-izes zero direction components into `dir` (`abs(rd) < 1e-6 → 1e-6·sign(rd+1e-9)`, lines 2314–2315) and uses `dir` for the **chunk-selection** DDA. ✓
- But `trace_world` then calls `march_chunk(ci, ro, rd, ...)` **with the original raw `rd`** (`common.wgsl:2339`) — zeros intact. The same raw-`rd` handoff exists at the other two world-level call sites: `trace_local` (`common.wgsl:2135`) and `shadow_ray_t` (`common.wgsl:2238`).
- Inside `march_chunk`, the per-step DDA computes `inv = 1.0 / rd` (`common.wgsl:1381`) and `tn = (bounds - ro) * inv` (`common.wgsl:1839`). For a stationary axis: `inv.y = +inf`, `bounds.y = bm`, `in_i.y ≤ cell.y ≤ ro.y` (because `pos_dir.y = rd.y > 0` is false), so `tn.y = -inf` (or NaN when `ro.y` sits exactly on the boundary). The comparisons (1840–1848) then select the degenerate axis:
 - `tn.x ≤ tn.y && tn.x ≤ tn.z → false` ($\text{finite} \leq -\text{inf}$),
 - `else if tn.y ≤ tn.z → true` ($-\text{inf} \leq -\text{inf}$) → `t = -inf`, `axis = 1`.
- `t ≥ t_end` is false for $-\text{inf}$, so the step proceeds: `pos = ro + rd * (-inf) = (-inf, NaN, NaN)` ($0 \times -\text{inf} = \text{NaN}$) → `cell` garbage → range check breaks the loop → **miss**.

The primary hit at $t=3.5$ works only because the entry cell itself is solid (the hit is taken before the first step). The first DDA *step* is where every axis-aligned ray dies. This affects any caller that can produce an axis-aligned direction: glass transmission (`trace_world`, `skip_glass`), water legs, secondary reflection rays derived from axis-aligned surface normals, and `shadow_ray_t` for an axis-aligned sun.

Fix (one line, matching CLO2's own approach): pass the epsilon-ized direction into the chunk marcher — `march_chunk(ci, ro, dir, ...)` — or epsilon-ize inside `march_chunk` itself (the latter also protects `march_chunk`'s other direct callers, incl. the fixture's `primary` leg and resolve's secondary rays). Hit positions shift by $\leq 1e-6 \cdot t$ laterally; `dir` has length $1+O(1e-12)$, so `t` is unchanged to ~ 7 digits.

1.2 `shadow_ray_t` omits the water-surface skip that `shadow_ray` has

`shadow_ray` (common.wgsl:~2091) skips the water **surface** interface under `SPEC_WATER_REPAIR` (the `rd.y > 1e-4 && ro.y < frame.sea_level` branch at common.wgsl:2098) so a submerged receiver does not count the water plane it looks up through as an occluder. `shadow_ray_t` (common.wgsl:~2221) — the tracer used by the decoupled shadow passes (`primary_shadow`) — has no such skip, and its own chunk-selection DDA (common.wgsl:2244) also lacks CLO2's axis guard. The two tracers are documented as answering the same question (the split is a scheduling change, not a semantic one; DSO2 claims control exactness against the monolithic build), so this is a divergence: decoupled shadows can reintroduce exactly the submerged-occluder spike `SPEC_WATER_REPAIR` was built to remove, for shadow rays that start or end at a water interface. It is invisible at the fixture vantages (none combine underwater receivers with the split path at a surface boundary) but is a real behavioral difference between the two arms of an A/B the docs call exact.

1.3 Dead code / dead switches (not bugs per se, but each is a trap)

Where	What
common.wgsl:2199 <code>SOFT_TAPS</code>	Pipeline override with no reader left after DSO2 replaced the 3-cone-tap soft shadows with 1 ray + bilateral blur. It is still supplied by the override table (mod.rs:4060, 3/7 by HQ flag), and an orphaned comment at common.wgsl:2263 ("The penumbral verdict: the centre bit plus <code>SOFT_TAPS</code> cone taps...") still describes the removed function — it now hangs above the water section. Costs a specialization dimension for nothing.
mod.rs:3002	The monolithic (non-split) resolve branch is unreachable: <code>FLAG_HI_SHADOW_PASS</code> is force-OR'd into the spec key (mod.rs:2777, scene.rs:500), so the <code>== 0</code> guard at 3002 is always false, and <code>let shadow = true;</code> is hardcoded at mod.rs:4075. Dead but maintained code.
config.rs:898 <code>--water-mottle</code>	Parsed, then force-zeroed with a warning (WA01). The flag is dead, but its help text and three docs still describe it as live (see §4.2).
<code>menu.rs</code> <code>Setting::Iso</code> <code>lateGlass</code> / <code>Setting::ShadowPass</code>	Enum entries and test coverage remain, but no menu row exposes them (§4.3). <code>Iso</code> <code>lateGlass</code> is still a <i>live</i> A/B in scene.rs:499 that the user can no longer reach from the UI.

2. Stale in-code comments (each contradicts code within reading distance)

#	Location	Comment says	Code says
2.1	mod.rs:3673 (<code>shader_source</code>)	" <code>common.wgsl</code> and the four pass files"	Six pass files are concatenated (tile_select, march, resolve, taa, shaft, shadow)
2.2	mod.rs:3694–3696 (<code>ENTRY_POINTS</code>)	" Every entry point <code>Renderer::new</code> builds a pipeline for, in dispatch order. <code>shaderstats</code> walks this list"	<code>Renderer::new</code> also builds <code>primary_shadow</code> , <code>shadow_horizontal</code> , <code>shadow_vertical</code> (mod.rs:4081–4082) — all three dispatched (mod.rs:~3030–3053), all three validated by naga (14 compute entry points = 11 listed + 3 missing). <code>shaderstats</code> therefore reports no register count for the three shadow pipelines , which is exactly the failure the comment says the list prevents.
2.3	timer.rs:22	"14 is the last pair <code>MAX_QUERIES</code> has room for"	<code>MAX_QUERIES = 22</code> ; pairs exist at 16 (<code>SHADOW_RAY</code>), 18 (<code>SHADOW_H</code>), 20 (<code>SHADOW_V</code>). 20 is the last pair.
2.4	mod.rs:3508	"Y is exact rather than toroidal because <code>WORLD_HEIGHT / SPACING</code> is 64 on the nose "	<code>WORLD_HEIGHT=512</code> , <code>SPACING=4</code> → 128 ; the assert 60 lines below (mod.rs:3573) pins <code>PROBE_DIM_Y == 128</code> .
2.5	mod.rs:3537	"...probe field: 12 MB against 3.9 GB of idle VRAM"	The comment 38 lines below (mod.rs:3575) says the field is 128×768×128 RGBA16F ≈ 100.7 MB (= 96 MiB, roadmap L5's own table).
2.6	probe.rs:123	occlusion "stored at 8-block spacing "	<code>SPACING = 4</code> (probe.rs:107).
2.7	probe.rs:437–438	column march "shared by the eight probes stacked in it... the difference between 20,480 height samples per chunk and eight times as many"	<code>PER_AXIS = 16</code> → 16 probes per column, 256 columns/chunk → 81,920 height samples (4× the old 20,480), per-probe-naïve = 16×.
2.8	probe.rs:693	"the chunk's 20,480 height samples stay 20,480"	81,920 at SPACING=4.
2.9	probe.rs:105–106 (the SPACING header itself)	" 8x the bake on the workers, 8x the VRAM"	VRAM 8× ✓ (12→96 MiB). Bake: the dominant cost is <code>WorldGen::height</code> calls, which scale with <i>columns</i> (area) = 4× (20,480→81,920); roadmap L5's own hardware measurement says 13,915 vs 3,256 μs = 4.3× , not 8×.

Items 2.4–2.9 are one cluster: batch L5 (`SPACING` 8→4, roadmap.md:1021) updated `probe.rs`'s header, the cache-size arithmetic (probe.rs:~306, 96 KB/288 KB/384 KB ✓), the render asserts, and roadmap — but missed the six places above, including two inside the very file that defines the constant.

3. Doc claims contradicted by current code (present-tense statements)

3.1 The L5 probe-spacing cluster (docs side)

The code is self-consistent at SPACING=4 (compile-time asserts pin `PROBE_DIM_XZ == 128`, `PROBE_DIM_Y == 128`; field = 128×768×128 RGBA16F = 96 MiB ≈ 100.7 MB; per-chunk residency 192 KB; 256 columns × 16 azimuths × 20 steps = 81,920 height samples/chunk). These present-tense doc statements are all from the SPACING=8 era:

Location	Stale claim	Actual
CLAUDE.md:146 (<code>--no-probe-cube</code> row)	"The field is one 3D texture of six stacked slabs, 8 blocks per probe "	4 blocks per probe
CLAUDE.md:151, :549	"the chunk's 20,480 height samples stay 20,480"	81,920
CLAUDE.md:154 (<code>--probe-tap</code> row)	"is 64 x 384 x 64 since batch 57" (present tense)	128×768×128 since L5
CLAUDE.md:505 (scarcity table — the canonical current inventory)	"probe field 12.0 ... ~ 142 MB ... 64 x 384 x 64 Rgba16Float is exactly 12,582,912 bytes ... 96.5% free "	probe field 96 MiB; total ≈ 231 MB; 128×768×128 = 100,663,296 B
PERF.md:332	" 64 probe columns x 16 azimuths x 20 steps = 20,480 <code>WorldGen::height</code> evaluations per chunk"	256 columns → 81,920
PERF.md:337	" VRAM: 12 MB , one 64 x 384 x 64 Rgba16Float 3D texture... 24 KB of that per resident chunk"	96 MiB; 192 KB per chunk
PERF.md:376	" VRAM is unchanged at 12 MB "	unchanged <i>by batch 58</i> — but reads as current; superseded by L5
docs/shading.md:390	"The field itself is no longer 128^3 — it is 64 x 384 x 64 "	it is now 128×768×128 (ironically back to 128 in XZ)
docs/shading.md:867	"The chunk's 20,480 height samples stay 20,480"	81,920
docs/research-review.md:848	"probe lattice is 64 x 384 x 64 → correct "	the review's verdict is now false

(PERF.md:276/:355/:1822 and ledger batch 57/58/60 rows are explicitly historical and fine under the repo's own "vantage" convention; the rows above are not marked historical or sit in canonical current-state tables. CLAUDE.md:154 even carries a "re-measure before quoting" hedge, but the dimension sentence itself is stated as present fact.)

3.2 docs/gpu.md SPEC_MASK section — contradicted by SpecPipes

gpu.md says "march and resolve are the only passes with more than one pipeline" and "SpecPipes holds march and resolve and nothing else". The struct (mod.rs:1730–1742) holds seven fields: `primary_shadow`, `shadow_blurs[2]`, `glass_resolve`, `water_resolve`, `march`, `resolve`, `water_sec` — five additions from batches 38b/81/DS02 that the doc never absorbed.

3.3 .gitattributes vs the tree, and vs pitfalls.md

- `.gitattributes:4–8` lists **three** CRLF strays: `src/lib.rs`, `src/stats.rs`, `docs/batch-11-godrays.md` — the third file **does not exist** (batch 11's doc was merged into `docs/sky.md`; `ledger.md:25` links `batch-11-godrays` → `sky.md`), and it claims "batch 22 re-verified that this list is still complete and still three".
- `docs/pitfalls.md` (line-ending map) lists only **two** strays ("and two strays — `src/lib.rs`, `src/stats.rs`"). The two files contradict each other; at most one can be right, and the missing file makes `.gitattributes` the wrong one.

3.4 Water-mottle: the flag is dead but four places still sell it

WA01 (BATCH-LOG) retired `--water-mottle`: `config.rs:898` force-zeroes it with a warning, and the struct default is 0.0 (`config.rs:666`). Still documented as functional:

- `config.rs:18` (struct doc: "`--water-mottle 0.2` is the control and reproduces the pre-batch-26 build bit for bit"), `:665` (comment), `:1155` (help text: "ships 0; 0.2 = pre-batch-26");
- `CLAUDE.md:117` control-table row: "`--water-mottle 0.2` | the pre-batch-26 sea | free (atlas constant)";
- `docs/water.md:436` control table — which contradicts **the same file's line 118**: "The water layer ships flat (`--water-mottle` 0) since batch 26".

Meanwhile `tests/textures.rs` pins `WATER_MOTTLE_PRE21B = 0.20` only as the historical constant, and the default-0.0 guard passes — so the code is consistent with WA01 and only the docs/help lie. Running the documented control is a silent no-op (warning to stderr, value discarded).

3.5 --probe-sun does not exist

`docs/lessons.md:73` and `docs/shading.md:1060` quote the flag `--probe-sun` ("`--probe-sun` at `PROBE_SUN_GAIN` 0.10"). The parser (`config.rs`) knows only `--no-probe-sun` and `--probe-sun-high` (`config.rs:1033`). Running the documented command hits `UNKNOWN_ARG_MARK` and exits.

3.6 Menu / experiments: docs claim 7 switches, the menu has 5

- `MENU-NOTES.md`, `EXPERIMENTS.md` and `README` describe 7 experimental switches and a "Render lab" with rows for `IsolateGlass` and `ShadowPass`.
- `menu.rs` `rows()` / `items()` expose 5; no row constructs `Setting::IsolateGlass` or `Setting::ShadowPass` (enum entries still exist; `tests/settings_menu.rs` pins no row count, so nothing catches it).

- `ShadowPass` is legitimately retired (DSO2 force-enables it, `--no-shadow-pass` warns), but `IsolateGlass` **is still a live A/B** (scene.rs:499 sets the flag from the setting) that the UI can no longer toggle — a functional regression against the docs, not just a doc drift.
- EXPERIMENTS.md additionally still describes the **pre-DSO2** shadow experiment: visibility "stored as full-resolution **f32 values in a storage buffer**... not quantized to eight bits" (EXPERIMENTS.md:55) and "**4 bytes per internal pixel**: approximately **7.91 MiB at 1920×1080**" (:79), plus lazy allocation and optional pipelines, and lists retired `--no-shadow-pass` as the revert. Reality (DSO2, verified in code and in `validation/check_decoupled.py`'s own bind groups): `R8Unorm` write-only storage **texture**, 8-bit quantized (the script asserts mask values are exactly 0/255), always allocated, 7 B/px all-in (~14.6 MiB at 1920×1080), all pipelines built unconditionally, visibility = 1 raw ray (`primary_shadow`) + screen-space bilateral blur (`shadow_horizontal` / `shadow_vertical`).

3.7 Soft-shadow help text predates DSO2

The `soft_shadows` field doc in `config.rs:117–121` still describes batch-102a's design as current: "penumbra — **three cone taps** around the centre shadow tap... with the tap budget scaled by blocker distance" (the experiment DSO2's own docs record as *not shipped*: +4.545 ms, 13× its bar, rejected). The `--help` text repeats it (`config.rs:1237`). The shipped implementation is one raw shadow ray (`primary_shadow`) + screen-space bilateral blur (`shadow_horizontal` / `shadow_vertical`, gated by `SPEC_SOFT_SHADOWS` at `mod.rs:4045` → `shadow.wgsl:71/99`); `--soft-shadow-hq` now widens the blur radius (`SHADOW_RADIUS_LIMIT` 4→8), not a tap count. SURFACE-REPAIRS.md:15/:67–74 still quote "four samples total / eight total", but that file carries a superseded-by-BATCH-LOG banner (line 1), so the `config.rs` doc + help are the live errors.

3.8 `pitfalls.md` "Known warts" contradicts ledger batch 102

`pitfalls.md`: "harness's own argument parser **silently ignores unknown arguments** (`Opts::parse` ends in `_ => {}`)". Ledger batch 102 (driver row): the parser is now **strict** — "unknown `--flag` names the vocabulary and exits 2 — **the paragraph was amended in place**". It was not amended; both statements cannot be true. (`src/bin/harness.rs` is absent from this dump — see §5 — so the code itself cannot arbitrate here, but the ledger explicitly claims the doc edit happened and it demonstrably did not.)

3.9 `pitfalls.md` references a doc family that no longer exists

"each `docs/batch-N` file holds its own batch's misses" — there are **no** `docs/batch-*.md` files in the tree (content lives in the topic docs; ledger.md:25 shows the merge pattern). Same root as §3.3's phantom `batch-11-godrays.md`.

4. Test-suite findings

4.1 `lod_stream` (6/9 fail) and `shadow_grid` (4/4 fail) — proven environmental, not logic bugs

Both targets fail only via their `settle()` helper ("world never settled", tests/lodstream.rs:36, tests/shadowgrid.rs:44): a budget of **6000 iterations** of `update` + 200 µs sleep ≈ **1.9 s wall** on this 2-core sandbox. Evidence they are throughput artifacts, in three independent runs:

- Instrumented debug repro:** streaming progresses monotonically (no thrash): requests issue coarsest-LOD-first, `in_flight` pins at the 64 cap, world histogram grows steadily (LOD4/3 fill first, LOD2 at ~70 chunks/s) — it simply runs out of budget with ~677 builds pending. `World::insert_lo` `cal` has no reject path; `ChunkKey::parent()` uses arithmetic shift (correct for negatives); unload bookkeeping is clean.
- Release, instrumented, 200k budget:** the `lod_stream` world **settles at iteration 20,669 (6.4 s)** — `total_drained` 922 ≈ 840 selected + ancestor redundancy, world size monotone to 840. No oscillation anywhere.
- Release, `shadow_grid`, 200k budget: all 4 tests pass** (27 s) — including every grid/yaw-independence assertion.

Verdict: the streaming/LOD logic is correct; the tests' iteration budget makes them **wall-clock-fragile on slow or few-core machines** (they pass on the author's 16-thread hardware; ledger batch 94 already recorded one of these, `transitions_are_symmetric_and_never_uncover`, as a pre-existing failure — consistent with a machine-dependent budget, not a regression). A time-based budget, or a `settle()` that also accepts "monotone progress", would make them robust. **Every other test target passes** (46 integration targets + lib + main units; 25 suites green in the aggregate run plus the 14 targets run individually after cargo stopped at `lod_stream`; doc-tests: 0). HANDOFF's CLO2 claim "all 48 test targets pass a release run" is consistent with the author's hardware and with everything measured here except the two budget-bound targets.

4.2 `tests/bins.rs` silently skips when `src/bin/` is absent

By design it "skips when the bins are missing" — which means `cargo test` can never flag the absence of the three documented binaries (harness, probe, shaderstats). Combined with §5's export omission, a working-from-dump session gets a green suite over a tree missing 3 binaries that 10+ doc locations reference as present. (In this dump the absence is the exporter's fault, not the repo's — but the silent-skip design hides exactly this class of gap.)

5. The dump itself: `gemini-code-1789442870285.py` omission rules explain all 17 "broken" links

The export script at the repo root generated `codebase_dump.txt` with:

- `IGNORE_DIRS` containing `"bin"` → `src/bin/harness.rs`, `src/bin/probe.rs`, `src/bin/shaderstats.rs` were dropped even though they are ordinary text sources (the pattern was meant for build-output dirs). Consequence: `cargo run --bin harness` etc. cannot work from this tree, `test s/bins.rs` skips, and 4 links in docs/ledger.md dangle.
- `IGNORE_EXTS` containing `.png/.jpg/.lock` → all 57 screenshots, the `look-reference-*.png` figures and `Cargo.lock` dropped (Cargo.lock was regenerated locally by cargo; harmless). `docs/research/` held only binaries (PDF/docx per pitfalls.md) and vanished wholesale.

All 17 broken markdown links found by the link audit are exactly these omissions (5 README screenshots, 4 ledger `src/bin` refs, 3 docs/lod.md batch-64 PNGs, 1 research-review → research/, 4 look-reference PNGs in roadmap.md/sky.md). **None is a repo defect** — though the over-broad `"bin"` rule is a defect of the export tooling, and `screenshots/README.md`'s index ("Tracked on purpose... every file here") describes files the dump cannot carry.

6. Verified-correct (bounded sample of the things most likely to be wrong)

To keep the report honest about coverage, these were checked and are **clean**:

- `cargo check --all-targets` (incl. benches): zero errors/warnings; `clippy`-style lints not re-run.
- **All WGS� compiles** under naga 30 with `Capabilities::all()`: combined module (14 compute entry points), blit vs/fs, ui vs/fs.
- `GpuFrame` (352 B) and `GpuChunk` (96 B) Rust↔WGS� field-for-field identical; group(3) bindings 1–5 consistent across shadow.wgsl and mod.rs; `BIND_GROUP_0_ENTRIES=22` matches common.wgsl bindings 0–20 + probetex; *timer query indices all < 22; dispatch sizes vs workgroup sizes consistent (resolve 16×8, taa 8×8, tileselect 8×8, water_sec span/8)*.
- All `FLAG_*` bit values and bit positions (0–27 + 31) identical between Rust and WGS�; `SPEC_*` override defaults match the control table; `shadow_guide` decode bit-identical to resolve's; `make_key` packing (depth:24↑40, normal:3↑37, chunk:13↑24, micro in bits 6–7 of each voxel byte) matches every decoder.
- DSo2/WAO1/GLO1/HKO1 handoff claims spot-checked against code: `range_gate = (1-smoothstep(p(48,96,t))·smoothstep(0.08,0.20,|rd.y|))` (resolve.wgsl:2247) ✓; `WaterCtx.roughness` optical blending (:2335) ✓; glass normal flip toward view, transmission range `max(GLASS_TRANSMIT_DIST, frame.far)` with `skip_water=false`, glass-specific `SPEC_GLASS_REFLECT` trace with water branch (resolve.wgsl:2423–2551) ✓; `SHADOW_RADIUS_LIMIT` 4/8-HQ ✓; `SOFT_TAPS` dead as documented ✓.
- CLO2's `trace_world` axis guard exists and is correct **as far as it goes** (see §1.1 for where it stops).
- Every engine CLI flag documented in CLAUDE.md's control table exists in the parser (sole exception §3.5); README keybinds (F3–F7, Esc, hotbar 1–0) match app.rs:618–628; `PROBE_SUN_GAIN` 0.10/0.25, `SUN_ANGLE`, `SOFT_MISS_DIST`, penumbra constants, LodConfig/Streamer defaults all match their docs.

- `tests/docs.rs` link guard passes (its exemptions — png/jpg/research/src-bin — are precisely the dump's omissions, §5).
- The recorded `validation/decoupled-results.txt` PASS run and `native-glass-results.txt` "cargo not found" are consistent with this environment; `check_glass_transport.py`'s failure is now fully explained by §1.1.

7. Summary by severity

Severity	Findings
Functional bug	§1.1 axis-aligned <code>march_chunk</code> miss (explains the recorded glass fixture failure; fix = pass epsilon-ized dir)
Latent divergence	§1.2 <code>shadow_ray_t</code> missing the water-repair skip
Contract violations in code	§2.2 <code>ENTRY_POINTS</code> omits 3 built+dispatched pipelines; §2.1/2.3 stale counts
Current-state doc errors	§2.4–2.9 + §3.1 (probe L5 cluster, 10 locations), §3.2 gpu.md, §3.3 gitattributes/pitfalls, §3.4 water-mottle ×4, §3.5 <code>--probe-sun</code> , §3.6–3.7 menu+EXPERIMENTS+soft-shadows, §3.8–3.9 pitfalls
UI regression vs docs	§1.3/§3.6 <code>IsolateGlass</code> live but unreachable
Test robustness	§4.1 settle-budget fragility (proven environmental here); §4.2 silent skip
Tooling	§5 export script drops <code>src/bin</code> + images + lockfile
Not errors	lodstream/shadowgrid logic, all 17 broken links (dump artifacts), everything in §6